Greedy Algorithm

Sort: sort jobs in non-decreasing order, sort jobs by finish times so that $f_1 \le f_2 \le \cdots \le f_n$

Initialize: define __ as__, create solution set $A \leftarrow \emptyset$, record the end time of the last selected task $last \leftarrow 0$

Iteratively select: Traversing the array/ sorted __, check if __ then __. **Output:** calculate__, return

We **proof** greedy algorithm is optimal by contradiction. →Greedy is optimal.

1. Assume there exists an optimal solution (OPT) with a __ greater than the one found by greedy ($GREEDY$).
2. Let $G_1(OPT_1)$, $G_2(OPT_2)\ldots, G_i(OPT_i)$ denote the set of __ selected by $GREEDY(OPT)$.
3. Let's consider the first operation where OPT and GREEDY differ. Find i be the first value that
   $G_1 = OPT_1, \ldots,\ G_{i-1} = OPT_{i-1},\ G_i \ne OPT_i$, [或者$G_i \in OPT \backslash GREEDY, G_j \in GREEDY \backslash OPT$]
4. By definition of $GREEDY$,
   a) it has included all possible students that satisfy the inequality, which means $OPT_{max} \le G_{max}$
   b) **"Exchange argument":** replacing x with y in OPT does not decrease(increase) total revenue. $\Delta_1 - \Delta_2 \ge 0$. Repeatedly exchanging such pairs transforms OPT_k to G_k without reducing(increasing) revenue and still feasible, which contradicts $OPT > GREEDY$.
   c) **"Cut lemma":** When e^* was added by the algorithm, it connected two components $C1$ and $C2$, where $C1, C2 \subset V$. OPT must contain a path P connecting $C1$ and $C2$, which includes a road e crossing the cut $(C1, C2)$. Since the algorithm chooses the smallest edge across this cut, $d(e^*) \le d(e)$, where $d()$ is the distance of connections. Replace e with e^* in OPT. The new set $OPT' = OPT \setminus \{e\} \cup \{e'\}$ is still connecting all cities, and its total distance satisfies: $d(OPT') = d(OPT) - d(e) + d(e') \le d(OPT)$. [旅游城市最小生成树]
   This contradicts the assumption that OPT is strictly better than $GREEDY$.

课程：Interval Scheduling(f_i) / Scheduling to Minimize Lateness(d_i)/ Minimum Spanning Tree($w(e)$)
Single-link k-clustering(删除最小生成树中 k - 1 条权最大的边)/ Optimal Caching(Farthest-In-Future, FF)

高买低卖：不连续高买低卖可以拆分成连续的高买低卖 **优先队列** priority queue：每次从优先队列中移除两个最短的数组，合并为一个新数组，重新放入优先队列。将这两个数组的长度之和累加到总成本中。

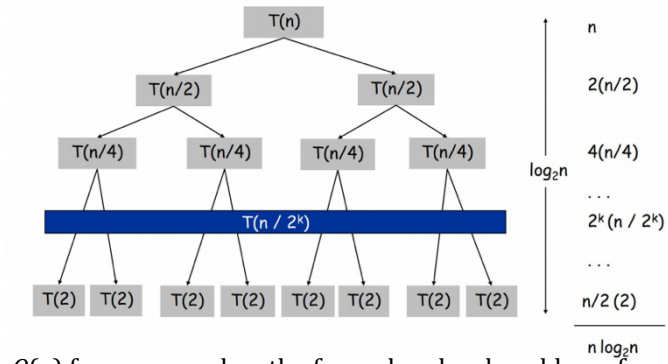
Divide and conquer

proof by recursion tree

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

Divide step: We divide the set/array of __ into two roughly equal halves. Recursively find __ for both the left and right halves until each subarray has a length of 1 or 0.

Combine step: For the left and right halves,
->find __ span both halves. [Compare it with ___ the left and right halves, and take the largest value as the maximum value of the current interval.]
->Recursively call merge sort to sort each of the two subarrays, then merge the two sorted subarrays into a single sorted array.[Merge sort]



Time complexity: The time complexity of combine step is $O(n)$ for sequence length of n, and each subproblem of left and right halves requires $T(n/2)$. The recurrence relation is: $T(n) = 2T(n/2) + O(n)$.
Thus the overall time complexity is $O(n \log n)$.

binary search: We start by checking the middle element of the array. If $A[\,n/2\,] =$ _ , we have found the desired index. Otherwise, we can recursively search the left half if $A[\,n/2\,] >$ _or the right half if $A[\,n/2\,] <$ _ when the array is ascending, vice versa.

quickselect: Randomly choose a pivot element from Δ.→Partition Δ into three subarrays: elements smaller than the pivot, equal to the pivot, and larger than the pivot.→Recurse only into the subarray that contains the m-th element. On average, QuickSelect runs in O(n) time, which is more efficient than full sorting $O(n \log n)$

课程：Closest Pair of Points[$\Theta(n\log n)$或$\Theta(n)$]/ Integer Multiplication [$\Theta(n^2) \to \Theta(n^{\log_2^3})$]/
Matrix Multiplication [$\Theta(n^3) \to \Theta(n^{\log_2^7})$]/ Mergesort [$\Theta(n\log n)$]/ FFT [$\Theta(n\log n)$]

最大连续子和: 例子/ **逆序数**：递归计算左、右子序列的逆序数；计算跨子序列的逆序对；合并累加逆序数

Dynamic Programming e(剩余图)=f(e^R)+c(e)-f(e)

State Definition: Define __as__ → **State Transition**: Boundary conditions; case1; case2; transition equation

Weighted Interval Scheduling: O(nlogn)

$p(j)$ = largest index $i < j$ such that job i is compatible with j, $OPT(j) = \max\{v_j + OPT(p(j)), OPT(j-1)\}$

Knapsack Problem: $\Theta(nW)$ Not polynomial in input size! "Pseudo-polynomial." NP-complete

$$OPT(i,w) = w - w_i: \max\{opt(i-1,w), v_i + opt(i-1, w-w_i\}; w_i > w: opt(i-1,w); i = 0: 0.$$

RNA Secondary Structure: $O(n^3)$ 子串长度*子串起始位置*遍历 t

$\max\{\max_{t\in T}\{1 + OPT(i,t-1) + OPT(t+1,j-1)\}, OPT(i,j-1)\}$ T:$i \le t < t-1, b_t, b_j$ can be pairs

Sequence Alignment: $\Theta(nm)$ → O(m+n) 无法恢复具体比对结果

$$\text{cost}(M) = \underbrace{\sum_{(x_i,y_j)\in M} \alpha_{x_i y_j}}_{\text{mismatch}} + \underbrace{\sum_{i:x_i \text{ unmatched}} \delta + \sum_{j:y_j \text{ unmatched}} \delta}_{\text{gap}}$$

$$OPT(i,j) = \begin{cases} j\delta & \text{if } i=0 \\ \min\begin{cases} \alpha_{x_i y_j} + OPT(i-1, j-1) \\ \delta + OPT(i-1, j) \\ \delta + OPT(i, j-1) \end{cases} & \text{otherwise} \\ i\delta & \text{if } j=0 \end{cases}$$

Shortest Paths: Dijkstra 算法负权重×→重加权法：长边短边×→OPT(i, v)为从 v 到 t、用不超 i 条边的最短路长。

$OPT(i,v): i = 0, v \ne t: \infty;\ v = t: 0;\ other: \min\{OPT(i-1,v), \min_{(v,w)\in E}\{opt(i-1,w) + c_{vw}\}\}$ ，无负成本环 $OPT(n\text{-}1)$

零钱兑换: $OPT(i) = \min\{OPT(i), OPT(i-c[j]) + 1\}$, initialize $OPT(0) = 0, OPT(i) = inf, i - c[j] < 0$

兑换方案: $OPT(i,j)$用前 i 种凑出 j. $j - c(i) < 0: OPT(i-1,j)$, $j - c(i) = 0: OPT(i-1,j) + 1$

$j - c(i) > 0: OPT(i-1,j) + OPT(i, j-c(i))$

打家劫舍: k=1: M_1; k=2:$\max\{M_1, M_2\}$; $k \ge 3: \max\{f(k-1), M_k + f(k-2)\}$

棋盘礼物最大价值: $i = j = 1: M_{11}$; $i = 1, j > 1: M_{1j} + f(1, j-1)$; $j = 1, i > 1: M_{i1} + f(i-1,1)$;

$i > 1, j > 1: \max\{f(i-1,j) + M_{i,j}, f(i,j-1) + M_{i,j}\}$

混插字母: $M[i, j]$ = T if and only if ($M[i-1, j]$ = T and $A[i] = C[i+j]$) or ($M[i, j-1]$ = T and $B[j] = C[i+j]$).

双色打印: $s[i] \ne s[j]: opt(i,j) = \min_{k=i}^{j-1}\{opt(i,k) + opt(k+1,j\}$; $s[i] = s[j]: opt(i, j-1)$

卖石头: $opt(i,j) = \max\{P_{i,j}, \max_{1\le k<i}\{opt(k,j) + opt(i-k,j) - j\} + \max_l\{opt(i,l) + opt(i,j-l) - i\}\}$

切割布料: $opt(j) = \max_{1\le i<j}\{opt(j-i) + price[i]\}$

交易股票 j 天不超过 i 次: $opt(i,j) = \max\{opt(i,j-1), \max_{1\le m<j}\{opt(i-1,m) + pric[j] - pric[m]\}; \max_i$

插删替单词匹配: $si = sj$: $\max\{opt(i,j-1), opt(i-1,j), opt(i-1,j-1) + 1\}$; $si \ne sj: opt(i-1,j-1)$

Network Flow adjacent 相邻的 vertex 顶点 Residual Graph 剩余图 integrality theorem 整数流定理

The augmenting path is __. The capacity of this path is min(, ,)= Ford-Fulkerson: $O(mnC)$ 指数级

改进：外层循环 $O(\log C)$次，每次内层循环 $O(m)$次增广，每次增广需 $O(m)$时间，总时间复杂度为$O(m^2 \log C)$。

Network construction: Node set→Edge and capacity set: a source s and a sink t;

Create a directed edge from s to __, with the capacity of the edge set to __

Bipartite Matching(人挑狗，一人一只) 增加 s,t, $(s,L)\leftarrow 1$ $(L,R)\leftarrow\infty$ $(R,t)\leftarrow 1$

Edge Disjoint Paths 每条边的容量设为 1，assign unit capacity to every edge，转化为最大流问题

Circulation Problems 增加 s,t, 对于 $v(d(v)<0)$: $(s,v)\leftarrow -d(v)$; $v(d(v)>0)$: $(s,v)\leftarrow d(v)$

问卷调查: (s-客户 i)←[c_i-c_i'], (客户 i-产品 j)←[0,1], (产品 i-t)← [p_i-p_i'], (s,t) ←[0,∞] 是否存在整数环流

图像分割: 对每个像素 i，$(s, i)\leftarrow a_i$；$(i, t)\leftarrow b_i$；$(j, i)\leftarrow p_{ij}$ 最小割容量$\sum_B a_i + \sum_A b_i + \sum p_i$为原最大化的补

棒球比赛: $(s, x-y)\leftarrow g_{xy}$剩余比赛 $(x-y, x/y)\leftarrow g_{xy}$; $(x,t)\leftarrow w_3 + r_3 - w_x$所有边容量饱和 则未淘汰

项目选择: if $(v,w)\in E$: (w,v) ←∞ If $p_v>0$: (s,v)← p_v; If $p_v<0$: (v,t)← -p_v; $p_s=p_t=0$; 最优项目子集为 A-{s}

网格最大礼物: checkerboard mincut(最小舍弃): (s, 黑)←p (白, t)←p (黑, 白)←∞ maxgift = 总价值-f(最大流/)

象棋最大骑士: 总-最多有多少对互相攻击的骑士 mincut: (s,白)←1; (黑,t)←1 (白,黑)if 攻击 ←∞ else 无边

工厂运货村庄: $(s, f_i)\leftarrow p_i$ $(f_i, v_j)\leftarrow c_i$ $(v_j, t)\leftarrow d_j$ check if maximum flow value $f = \sum d_i$

Asymptotic Notation A dominate B add/minus/multiple/divided by lower-order terms omitted

For *res*1, the outer/middle/inner loop is executed n times, which is $O(\,)$. The overall complex is __

The nested loops over indices i and j. / A __ step on an array of length n./ with <u>a binary search</u> inside.